MATRIXPOLICY FOR ROLE-AWARE OPTIMIZATION OF RATIONAL TRANSFORMER NONLINEARITIES

Anonymous authors
Paper under double-blind review

ABSTRACT

A useful LLM pretraining recipe moves the loss-time frontier by reaching the same validation loss with fewer tokens or less wall-clock time. We target this objective through the Transformer nonlinear sublayer, treating it as an optimizer-visible interface. MatrixPolicy with Rational Latent Basis (RLB) replaces a fixed scalar response with grouped rational responses that expose local gain, coefficient-activity, and matrix-balance statistics to a role-aware update. MatrixPolicy uses these signals to update only the sublayer input and output matrices, while rational coefficients and the rest of the Transformer remain on AdamW. Across DCLM, FineWeb-Edu, FineWeb, Dolma-sample, and C4 with a fixed 12-layer, width-768 decoder, MatrixPolicy reaches matched validation-loss targets with fewer tokens and lower measured arrival time than SiLU controls under the same protocol. It saves 19.5–22.8% target-arrival tokens versus SiLU with AdamW and 29.2–34.0% versus SiLU with Muon at 100M tokens, and 22.3–26.4% and 6.7–9.8%, respectively, at 300M tokens. [todo: Insert the larger-scale training result here.]

1 INTRODUCTION

Recent LLM optimizer papers frame practical efficiency through loss-time and compute-time frontiers, asking which training recipe reaches a target loss with fewer tokens, FLOPs, or wall-clock time (Shah et al., 2025; Liu et al., 2025; Semenov et al., 2025). This criterion also exposes an activation-design question. In a modern decoder block, the nonlinear sublayer is a learned matrix–nonlinearity–matrix map. The activation sets local gain and curvature, while the adjacent matrices choose and recombine the coordinates on which that response acts. Standard training pipelines leave this local structure implicit to the optimizer. Activation coefficients and adjacent matrices are updated as ordinary parameters, so matrix updates do not receive the gain, coefficient-activity, or role information generated inside the sublayer.

We make this interface concrete with Rational Latent Basis (RLB) and MatrixPolicy. RLB maps residual states into grouped latent coordinates, normalizes each group by its RMS radius, and applies a trainable real-pole-free P5/Q4 rational response. The resulting response-gain, coefficient-activity, matrix-pressure, and pair-balance signals are detached from the forward graph and passed to MatrixPolicy. MatrixPolicy uses them to update only the RLB input and output matrices; attention, embeddings, normalization parameters, rational coefficients, and ordinary Transformer tensors remain on AdamW. This local division lets the nonlinear sublayer expose the quantities it creates without changing the optimizer used for the rest of the model.

We evaluate target arrival by measuring the tokens and wall-clock time required to reach a fixed validation-loss target under the same model, corpus, and protocol. In the completed study, the same 12-layer, width-768 causal Transformer is trained on DCLM, FineWeb-Edu, FineWeb, Dolma-sample, and C4. The experiments include RLB-only controls and matched SiLU with AdamW and SiLU with Muon baselines under the same model, data, and measurement protocol. MatrixPolicy reaches shared validation-loss targets earlier at both 100M and 300M token budgets. [todo: Insert the larger-scale training result here.] The contribution is an activation design that exposes optimizer-visible matrix roles, a localized role-aware optimizer for the exposed matrix pair, and matched evidence that this interface reduces target-arrival tokens and wall-clock time across five pretraining corpora.

2 RELATED WORK

Activation design for Transformer nonlinear sublayers has moved from fixed pointwise curves to gated matrix sublayers. Early Transformer-style sublayers used fixed pointwise responses such as GELU or Swish/SiLU (Vaswani et al., 2017; Hendrycks & Gimpel, 2016; Ramachandran et al., 2017). GLU introduced multiplicative feature selection for language modeling (Dauphin et al., 2017), GLU variants such as SwiGLU improved Transformer sublayers in compute-matched settings (Shazeer, 2020), and large decoder models adopted SwiGLU together with pre-normalized Transformer components (Chowdhery et al., 2022; Touvron et al., 2023). These works identify the nonlinear block as the natural comparison point for RLB because the response is coupled to the matrices that prepare and mix it. RLB keeps that sublayer location while replacing the fixed curve with grouped trainable rational responses.

Choosing a trainable rational response builds on earlier evidence that low-degree quotients can serve as learnable nonlinearities. Padé Activation Units learned such quotients end to end as activation functions (Molina et al., 2020), rational neural networks gave approximation efficiency results for smooth-function classes (Boullé et al., 2020), and RAFT evaluated rational activation functions inside Transformer pretraining and fine-tuning (Fang et al., 2023). Recent theory strengthens the motivation by proving expressivity advantages for trainable low-degree rational activations over a broad set of modern fixed activations, including gated and Transformer-style nonlinearities (Tang & Townsend, 2026). RLB therefore follows the rational-activation lineage; MatrixPolicy tests the additional effect of making the surrounding matrices optimizer-visible.

Making the surrounding matrices optimizer-visible connects RLB to a fast-moving LLM optimizer literature. Adam and AdamW remain central baselines (Kingma & Ba, 2015; Loshchilov & Hutter, 2019), while state-efficient or diagonal adaptive methods such as Adafactor and CAME reduce optimizer memory or alter adaptive-state estimates (Shazeer & Stern, 2018; Luo et al., 2023). Other recent methods change the update rule itself. Lion, Schedule-Free methods, and cautious optimizers alter momentum, sign, or schedule behavior (Chen et al., 2023; Defazio et al., 2024; Liang et al., 2026), while Sophia, GaLore, and Adam-mini change curvature, low-rank, or per-block learning-rate structure (Liu et al., 2024; Zhao et al., 2024; Zhang et al., 2025). Practical-efficiency studies evaluate optimizers by the tokens, compute, or time needed to reach matched losses (Shah et al., 2025; Schlotthauer et al., 2025; Semenov et al., 2025; Wen et al., 2026). The matrix restriction in MatrixPolicy connects most directly to matrix-aware optimization. Shampoo uses tensor preconditioners (Gupta et al., 2018), SOAP stabilizes Shampoo-style preconditioning by running Adam in the preconditioner eigenbasis (Vyas et al., 2025), and Muon uses orthogonalized matrix updates for LLM training (Liu et al., 2025). Recent optimizer comparisons emphasize fair tuning, model-scale dependence, and setting-dependent tradeoffs (Zhao et al., 2025; Wen et al., 2026). MatrixPolicy follows this matrix-aware direction, with updates restricted to the RLB input/output matrix pair and conditioned on activation-derived sublayer statistics.

3 METHOD

RLB and MatrixPolicy are designed as a single interface. RLB defines the nonlinear sublayer and the detached summaries it exposes; MatrixPolicy consumes those summaries to update the two adjacent matrices. Rational coefficients and ordinary Transformer parameters remain on the AdamW path in the evaluated configuration.

Throughout the method we use

$$\mathrm{clip}(x, a, b) = \min\{\max\{x, a\}, b\}, \tag{1}$$

$$\mathrm{smoothstep}(a, b, x) = 3\omega^2 - 2\omega^3, \qquad \omega = \mathrm{clip}\left(\frac{x-a}{b-a}, 0, 1\right). \tag{2}$$

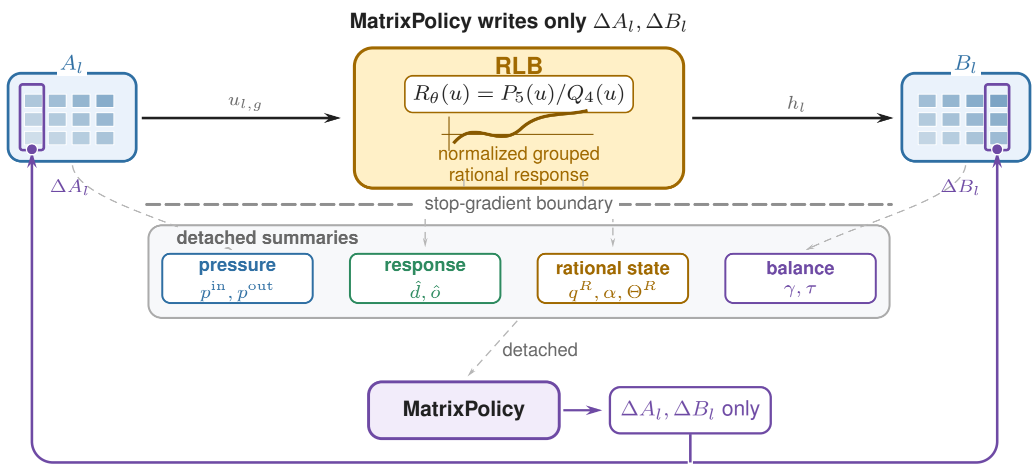

Figure 1: Activation–optimizer interface for RLB. In layer l, A_l maps the sublayer input into normalized rational groups and B_l recombines the rational features $h_l = \mathrm{concat}_{g=1}^{G} h_{l,g}$. Forward RLB statistics, gradient pressure/activity EMAs, and pair-balance measurements cross a stop-gradient boundary as policy inputs. MatrixPolicy reads these summaries and writes only ΔA_l and ΔB_l; rational coefficients are AdamW-trained and policy-observed, not MatrixPolicy update targets.

3.1 Rational Latent Basis

For layer $l \in \{1, \ldots, L\}$, let $x_l \in \mathbb{R}^d$ be the input to the nonlinear sublayer. RLB uses an input matrix $A_l \in \mathbb{R}^{H \times d}$ and an output matrix $B_l \in \mathbb{R}^{d \times H}$ with latent width $H = Gm$. Let $A_{l,g} \in \mathbb{R}^{m \times d}$ denote the row block of A_l, and let $B_{l,g} \in \mathbb{R}^{d \times m}$ denote the matching column block of B_l. In the implementation, A_l and B_l are the bias-free `in_proj.weight` and `out_proj.weight` of the nonlinear sublayer. The evaluated runs use an RMS floor $\epsilon_{\mathrm{rms}} = 10^{-6}$. Each group has a trainable P5/Q4 rational response

$$P_{l,g}(u) = \sum_{i=0}^{5} a_{l,g,i} u^i, \tag{3}$$

$$Q_{l,g}(u) = 1 + |b_{l,g,1}|\,|u| + |b_{l,g,2}|\,u^2 + |b_{l,g,3}|\,|u|^3 + |b_{l,g,4}|\,u^4, \tag{4}$$

$$R_{l,g}(u) = P_{l,g}(u)/Q_{l,g}(u). \tag{5}$$

Define the preactivation, RMS radius, normalized coordinate, and grouped rational feature by

$$z_{l,g}(x_l) = A_{l,g} x_l, \qquad \rho_{l,g}(x_l) = \sqrt{m^{-1}\|z_{l,g}(x_l)\|_2^2 + \epsilon_{\mathrm{rms}}}, \tag{6}$$

$$u_{l,g}(x_l) = \frac{z_{l,g}(x_l)}{\rho_{l,g}(x_l)}, \qquad h_{l,g}(x_l) = \rho_{l,g}(x_l) R_{l,g}(u_{l,g}(x_l)), \qquad h_l = \mathrm{concat}_{g=1}^{G} h_{l,g}. \tag{7}$$

The RLB forward map is the single block expression

$$y_l = B_l h_l = \sum_{g=1}^{G} B_{l,g} \rho_{l,g}(x_l) R_{l,g}\left(\frac{A_{l,g} x_l}{\rho_{l,g}(x_l)}\right). \tag{8}$$

The scalar functions $P_{l,g}$, $Q_{l,g}$, and $R_{l,g}$ are applied coordinatewise when their input is a vector. Since $Q_{l,g}(u) \geq 1$ for all real u, the implemented response has no real pole. The response is piecewise rational because of the absolute-value terms; derivative statistics use the autodiff convention $\mathrm{sign}(0) = 0$ and equations involving $R'_{l,g}$ are interpreted almost everywhere. Numerator and denominator coefficients are initialized by a SiLU fit on $[-5, 5]$ and are then trained by AdamW. The evaluated model uses only these groupwise P5/Q4 rational responses.

The forward map in equation 8 separates the normalized coordinate at which the rational response is evaluated from the group radius returned to the residual stream. This radius-factor form exposes the positive homogeneity behind the matrix-pair coordinate formalized in Appendix B. The floor $\epsilon_{\mathrm{rms}} > 0$ is an additive squared-RMS stabilizer. For fixed curves $R_l = \{R_{l,g}\}_{g=1}^{G}$, write this block map as $f_l(x; A_l, B_l, R_l)$. For any fixed input x such that $A_{l,g}x \neq 0$ for every group, if $\epsilon_{\mathrm{rms}} = 0$, then for any $\lambda \in \mathbb{R}^{G}_{>0}$ and $D_l(\lambda) = \mathrm{diag}(\lambda_1 I_m, \ldots, \lambda_G I_m)$,

$$f_l(x; A_l, B_l, R_l) = f_l(x; D_l(\lambda)A_l, B_l D_l(\lambda)^{-1}, R_l). \tag{9}$$

This positive matrix-pair rescaling is exact only in the unfloored map. In the actual training dynamics with the stabilizing RMS floor and optimizer state, MatrixPolicy uses the same direction as a bounded balance coordinate.

3.2 MatrixPolicy

MatrixPolicy acts only on the RLB matrix pair (A_l, B_l). In the evaluated optimizer, all policy inputs are detached summaries. During the forward pass, the RLB activation hook forms normalized coordinates and stores detached per-group response and derivative RMS summaries. After backpropagation, the wrapper reads the current gradients on A_l, B_l, and the rational coefficients, updates per-group EMA memories, and shares those memories with the matrix optimizer. The summaries are then consumed in implementation order. Before the optimizer step, MatrixPolicy rescales each group gradient by a role-specific multiplier $c_{l,g,\sigma}$, so the input matrix and output matrix can respond differently to derivative gain, response gain, pressure imbalance, and coefficient activity. During the matrix step, it applies a role/depth-scaled AdamW update and an early-training Muon substep to the same scaled gradient. After the child optimizers step, the wrapper applies a clipped reciprocal rescaling of $(A_{l,g}, B_{l,g})$ every five optimizer steps to move the matrix-pair RMS ratio toward a gain- and pressure-dependent target. The detached summaries therefore never change the forward computation directly; they control gradient scaling, matrix-step learning rates, Muon activation, and the post-step representative of the same RLB matrix pair.

The quantities below follow this order. Gradient-derived statistics are computed before parameter updates, while the pair-rescaling ratio is measured after the AdamW/Muon matrix branch. For any tensor V, define

$$\mathrm{rms}(V) = \sqrt{\mathrm{mean}(V^2)}, \qquad \mathrm{rms}_{\epsilon}(V) = \sqrt{\mathrm{mean}(V^2) + \epsilon}. \tag{10}$$

The constants $\epsilon_p > 0$, $\epsilon_{\mathrm{live}} > 0$, and $\epsilon_{\mathrm{probe}} > 0$ denote fixed additive floors in squared-RMS units. The same ϵ_p is reused as the implementation floor for matrix-ratio logs in equation 29; ϵ_{live} belongs to cached batch activation gains, and $\epsilon_{\mathrm{probe}}$ belongs to fixed-grid wrapper probes. The resulting pressure and activity scores are bounded update-control signals. Let $A_{l,g} \in \mathbb{R}^{m\times d}$ be the row block of A_l, and let $B_{l,g} \in \mathbb{R}^{d\times m}$ be the column block of B_l. The coefficient set $\Theta^R_{l,g}$ collects the trainable rational-response coefficient blocks present in group (l, g); in the evaluated groupwise P5/Q4 response this is the numerator and denominator coefficient pair. The instantaneous detached measurements are the input-matrix relative gradient p^{in}, the output-matrix relative gradient p^{out}, and the raw rational-coefficient gradient activity p^R:

$$p^{\mathrm{in}}_{l,g} = \frac{\mathrm{rms}_{\epsilon_p}(\nabla A_{l,g})}{\mathrm{rms}_{\epsilon_p}(A_{l,g})}, \qquad p^{\mathrm{out}}_{l,g} = \frac{\mathrm{rms}_{\epsilon_p}(\nabla B_{l,g})}{\mathrm{rms}_{\epsilon_p}(B_{l,g})}, \tag{11}$$

$$p^{R}_{l,g} = \left[\frac{1}{|\Theta^R_{l,g}|}\sum_{\theta\in\Theta^R_{l,g}} \mathrm{mean}((\nabla\theta)^2) + \epsilon_p\right]^{1/2}. \tag{12}$$

The same numerical floor is used in $p^R_{l,g}$, where the measured quantity is raw coefficient-gradient energy rather than a gradient-to-weight ratio. The displayed p values are evaluated at the current optimizer step; in the EMA recurrence, $p^r_{l,g,t}$ denotes that step-t value for role $r \in \{\mathrm{in}, \mathrm{out}, R\}$. MatrixPolicy does not use the instantaneous values directly. It stores detached EMA memories q^{in}, q^{out}, and q^R,

$$q^r_{l,g,t} = 0.95\, q^r_{l,g,t-1} + 0.05\, p^r_{l,g,t}, \qquad r \in \{\mathrm{in}, \mathrm{out}, R\}, \tag{13}$$

initialized from the first observed post-backpropagation values. The symbol q therefore denotes the policy memory of a detached measurement, not a rational denominator. The two contrasts used by later rules are

$$\pi_{l,g} = \log q_{l,g}^{\text{in}} - \log q_{l,g}^{\text{out}}, \qquad \alpha_{l,g} = \log q_{l,g}^{R} - \frac{1}{2}\left(\log q_{l,g}^{\text{in}} + \log q_{l,g}^{\text{out}}\right). \tag{14}$$

Each matrix pressure is an unsigned floored relative RMS-gradient proxy, so $\pi_{l,g}$ is signed only because it compares the input and output proxy memories. MatrixPolicy uses $\pi_{l,g}$ as a bounded pressure signal for redistribution, Muon attenuation, and pair balancing. The score $\alpha_{l,g}$ compares rational-coefficient activity with the geometric mean of the adjacent matrix-pressure memories; it is a bounded implementation contrast used for damping and rescale speed, not a scale-invariant physical ratio. The step subscript is dropped below when the time is clear.

The activation also provides batch-estimated curve gains. The RLB forward statistics hook subsamples activation inputs, forms normalized coordinates, and stores per-group response and derivative RMS summaries. MatrixPolicy reads the floored summaries

$$\hat{d}_{l,g}^{\text{live}} = \sqrt{\text{mean}\Big(R_{l,g}'(u_{l,g})^2\Big) + \epsilon_{\text{live}}}, \qquad \hat{o}_{l,g}^{\text{live}} = \sqrt{\text{mean}(R_{l,g}(u_{l,g})^2) + \epsilon_{\text{live}}}. \tag{15}$$

The mean is over sampled scalar normalized coordinates for group (l, g) across batch, sequence positions, and the m group coordinates. The derivative gain is the input-role proxy; the response gain is the recombination-role proxy. The latter excludes the group radius $\rho_{l,g}(x_l)$, so neither gain is a full feature scale or block-Jacobian preconditioner.

The detached summaries have distinct consumers. The live gains $(\hat{d}^{\text{live}}, \hat{o}^{\text{live}})$ enter only the gain term of the group-gradient multiplier. The pressure memory contrast $\pi_{l,g}$ enters role-opposed group gates, the Muon attenuation term, and the pair-balance target. The coefficient-activity contrast $\alpha_{l,g}$ damps group gates and the Muon branch when the rational response itself is moving strongly, while the wrapper uses it as a bounded rescale-speed factor. Finally, the post-branch matrix log-ratio $\gamma_{l,g}$ and target $\tau_{l,g}$ are used only by the wrapper's reciprocal pair rescale.

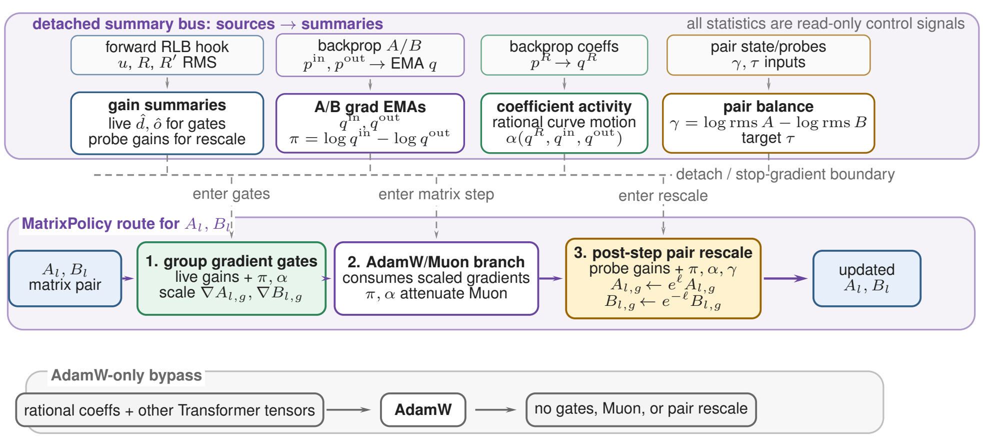

Figure 2: Detached-summary route for the evaluated MatrixPolicy system. The top lane shows where each detached summary family is produced; the middle lane shows where those summaries enter group-gradient gates, the AdamW/Muon matrix branch, and the post-step reciprocal pair rescale. Rational coefficients and non-RLB Transformer parameters remain AdamW-only.

Let t be the optimizer step, T the planned number of training steps, and $\xi_t = t/T$. A smooth gate $\phi_t = \text{smoothstep}(0.02, 0.30, \xi_t)$ turns on the group-gradient policy. The evaluated configuration enables this optional policy with gain, pressure, and activity strengths 0.20, 0.10, and 0.20, and clips group multipliers to $[0.75, 1.35]$. For positive group values v_j, define $\text{geomean}_{j=1}^{G}(v_j) =$

$\exp\Big(G^{-1}\sum_{j=1}^{G}\log v_j\Big)$. For group g and role $\sigma \in \{\mathrm{in}, \mathrm{out}\}$, the multiplier is

$$c_{l,g,\sigma} = c^{\mathrm{gain}}_{l,g,\sigma} c^{\mathrm{pressure}}_{l,g,\sigma} c^{\mathrm{activity}}_{l,g}. \tag{16}$$

$$c^{\mathrm{gain}}_{l,g,\mathrm{in}} = \left(\frac{\mathrm{geomean}_{j=1}^{G}\, \hat{d}^{\mathrm{live}}_{l,j}}{\hat{d}^{\mathrm{live}}_{l,g}}\right)^{0.20\phi_t}, \qquad c^{\mathrm{gain}}_{l,g,\mathrm{out}} = \left(\frac{\mathrm{geomean}_{j=1}^{G}\, \hat{o}^{\mathrm{live}}_{l,j}}{\hat{o}^{\mathrm{live}}_{l,g}}\right)^{0.20\phi_t}. \tag{17}$$

$$c^{\mathrm{pressure}}_{l,g,\mathrm{in}} = \exp(-0.10\phi_t\tilde{\pi}_{l,g}), \qquad c^{\mathrm{pressure}}_{l,g,\mathrm{out}} = \exp(+0.10\phi_t\tilde{\pi}_{l,g}). \tag{18}$$

$$c^{\mathrm{activity}}_{l,g} = \exp\left[-0.20\phi_t\, \mathrm{ReLU}\left(\frac{\alpha_{l,g} - 0.05}{0.45}\right)\right]. \tag{19}$$

where $\tilde{\pi}_{l,g} = \mathrm{clip}(\pi_{l,g}, -1.50, 1.50)$. Thus live derivative gain controls the input role, live response gain controls the output role, relative-gradient pressure acts with opposite signs on the two roles, and coefficient activity downweights groups whose rational curve is moving more strongly than the adjacent matrices. The final multiplier is recentered and clipped,

$$c_{l,g,\sigma} \leftarrow \frac{c_{l,g,\sigma}}{\mathrm{geomean}_{j=1}^{G}\, c_{l,j,\sigma}}, \qquad c_{l,g,\sigma} \leftarrow \mathrm{clip}(c_{l,g,\sigma}, 0.75, 1.35), \tag{20}$$

so the policy redistributes update scale across groups, separately for each matrix role, without changing the role-wise mean log multiplier before clipping.

The scaled matrix gradients are

$$\widetilde{\nabla} A_{l,g} = c_{l,g,\mathrm{in}}\nabla A_{l,g}, \qquad \widetilde{\nabla} B_{l,g} = c_{l,g,\mathrm{out}}\nabla B_{l,g}. \tag{21}$$

Let $\widetilde{\nabla} M_{l,\mathrm{in}}$ be the row-block concatenation of $\{\widetilde{\nabla} A_{l,g}\}_{g=1}^{G}$, and let $\widetilde{\nabla} M_{l,\mathrm{out}}$ be the column-block concatenation of $\{\widetilde{\nabla} B_{l,g}\}_{g=1}^{G}$. After this scaling, the RLB matrices are updated by a role/depth AdamW branch and a transient Muon branch. Define $\delta_l = (l-1)/(L-1)$ for $L > 1$ and $\delta_l = 1/2$ otherwise. The role/depth factors, AdamW multiplier, Muon attenuation, and Muon gate are

$$\varrho_{\mathrm{in}}(l) = \mathrm{clip}(1 - 0.50(\delta_l - 0.5), 0.55, 1.40), \qquad \varrho_{\mathrm{out}}(l) = \mathrm{clip}(1 + 1.00(\delta_l - 0.5), 0.55, 1.40), \tag{22}$$

$$s^{\mathrm{Adam}}_{l,\sigma} = \mathrm{clip}\left(3.0 \max\{0.10, 1 + 1.20(\varrho_\sigma(l) - 1)\}, 0.40, 4.0\right). \tag{23}$$

$$\Psi_{l,t} = \mathrm{clip}\left(\frac{1}{G}\sum_{g=1}^{G}|\pi_{l,g}|, 0, 1.50\right), \qquad \Omega_{l,t} = \mathrm{ReLU}\left(\frac{G^{-1}\sum_{g=1}^{G}\alpha_{l,g} - 0.05}{0.45}\right), \tag{24}$$

$$\psi_{l,t} = \mathrm{clip}\left(\exp[-0.30\Psi_{l,t} - 0.65\Omega_{l,t}], 0.10, 1.15\right). \tag{25}$$

$$\mu_{l,\sigma,t} = \mathrm{clip}\left(0.75\ \mathrm{smoothstep}(0.02, 0.12, \xi_t)[1 - \mathrm{smoothstep}(0.20, 0.36, \xi_t)]\varrho_\sigma(l)\psi_{l,t}, 0, 0.75\right). \tag{26}$$

For $M_{l,\mathrm{in}} = A_l$ and $M_{l,\mathrm{out}} = B_l$, let η_t be the base learning rate. MatrixPolicy applies an AdamW matrix step followed by any active Muon substep. The gate $\mu_{l,\sigma,t}$ scales the two learning rates; it is not a convex mixing coefficient for a single update direction:

$$\begin{aligned} \widetilde{M}^{t+1}_{l,\sigma} &= \mathrm{AdamW}_{\eta_t s^{\mathrm{Adam}}_{l,\sigma}(1-\mu_{l,\sigma,t})}(M^t_{l,\sigma}; \widetilde{\nabla} M^t_{l,\sigma}), \\ M^{t+1}_{l,\sigma} &= \mathrm{Muon}_{\eta_t\mu_{l,\sigma,t}}(\widetilde{M}^{t+1}_{l,\sigma}; \widetilde{\nabla} M^t_{l,\sigma}), \end{aligned} \tag{27}$$

Equation equation 27 is operational: AdamW and Muon keep separate states, use their configured decay and momentum terms, and both consume the step-t gradient. Here Muon is the `torch.optim.Muon` update on two-dimensional RLB matrices with momentum 0.95, five Newton–Schulz steps, and match-RMS learning-rate adjustment (Liu et al., 2025); it is inactive after all $\mu_{l,\sigma,t}$ decay to zero.

The surrounding on-policy wrapper's last MatrixPolicy-specific operation is a bounded positive rescaling of the two matrix blocks around each rational curve. It is evaluated after the AdamW/Muon matrix branch every five optimizer steps. Let $\mathcal{U} = \{u_k\}_{k=1}^{129}$ be a uniform probe grid over $[-5, 5]$, and define $\mathrm{rms}_{\mathcal{U}}(\varphi) = \sqrt{|\mathcal{U}|^{-1} \sum_{u_k \in \mathcal{U}} \varphi(u_k)^2}$. Fixed-grid probe gains describe curve shape for pair balancing; live gains in equation 15 describe the current batch distribution for group-gradient redistribution. The wrapper computes

$$\hat{d}_{l,g}^{\mathrm{probe}} = \sqrt{|\mathcal{U}|^{-1} \sum_{u_k \in \mathcal{U}} R'_{l,g}(u_k)^2 + \epsilon_{\mathrm{probe}}}, \qquad \hat{o}_{l,g}^{\mathrm{probe}} = \sqrt{|\mathcal{U}|^{-1} \sum_{u_k \in \mathcal{U}} R_{l,g}(u_k)^2 + \epsilon_{\mathrm{probe}}}, \tag{28}$$

$$\gamma_{l,g} = \log \mathrm{rms}_{\epsilon_p}(A_{l,g}) - \log \mathrm{rms}_{\epsilon_p}(B_{l,g}). \tag{29}$$

$$\tau_{l,g} = \frac{1}{2}\left(\log \hat{d}_{l,g}^{\mathrm{probe}} - \log \hat{o}_{l,g}^{\mathrm{probe}}\right) + 0.25\,\bar{\pi}_{l,g}, \qquad \bar{\pi}_{l,g} = \mathrm{clip}(\pi_{l,g}, -1.25, 1.25). \tag{30}$$

$$\chi_{l,g}^{\mathrm{rs}} = \exp(0.10\,\bar{\alpha}_{l,g}), \qquad \bar{\alpha}_{l,g} = \mathrm{clip}(\alpha_{l,g}, -1.25, 1.25). \tag{31}$$

$$s_{l,t} = 0.50\ \mathrm{smoothstep}(0.03, 0.35, \xi_t)\,\mathrm{clip}(1 + 0.15(\delta_l - 0.5), 0.75, 1.25), \tag{32}$$

$$\ell_{l,g,t} = \mathrm{clip}\left(\frac{1}{2}[\tau_{l,g} - \gamma_{l,g}] s_{l,t} \chi_{l,g}^{\mathrm{rs}}, -0.030, 0.030\right). \tag{33}$$

$$A_{l,g} \leftarrow e^{\ell_{l,g,t}} A_{l,g}, \qquad B_{l,g} \leftarrow e^{-\ell_{l,g,t}} B_{l,g}. \tag{34}$$

Here $\gamma_{l,g}$ is a floored matrix-RMS proxy. The first term of $\tau_{l,g}$ equalizes derivative and response probe gains in the positive log coordinate, the pressure term shifts the target boundedly, and χ^{rs} changes only the clipped rescale speed. Since equation 29 is floored, equation 34 is exact only in the nonzero, unfloored idealization; near the floor it is a clipped balance step. Optimizer-state buffers are scaled covariantly, with square-average entries using squared scales.

4 Experiments

4.1 Fixed-Scale Target-Arrival Study

The fixed-scale study uses the same 12-layer, width-768 causal Transformer on five corpora: DCLM/DataComp-LM (Li et al., 2024), FineWeb-Edu and FineWeb (Penedo et al., 2024), Dolma-sample (Soldaini et al., 2024), and C4 (Raffel et al., 2020). The 100M-token budget uses 3050 optimizer steps and the 300M-token budget uses 9150 optimizer steps, with three seeds per method/dataset pair. The main comparison keeps the architecture and data fixed while changing the activation and optimizer used in the nonlinear sublayer: MatrixPolicy is compared with SiLU controls and with RLB-only controls, where RLB-only uses the same RLB activation but no MatrixPolicy updates. Shared optimizer, evaluation-cadence, and throughput-measurement details are reported in Appendix A.1; broader optimizer comparisons beyond AdamW and Muon are summarized in Table 2.

Table 1: Target-arrival efficiency for the fixed 12-layer, width-768 Transformer. RLB denotes the same rational latent-basis activation without MatrixPolicy. At the hardest validation-loss target reached by every listed method in all three seeds, MatrixPolicy is always fastest; each token-saving column reports MatrixPolicy's savings relative to the method named below it.

			MatrixPolicy token savings (%)				Time to target (min)				
Budget	Dataset	Target loss	vs. SiLU AdamW	vs. SiLU Muon	vs. RLB AdamW	vs. RLB Muon	Matrix Policy	SiLU AdamW	SiLU Muon	RLB AdamW	RLB Muon
100M	DCLM	4.55	20.7	32.4	20.7	34.3	12.8	20.8	26.0	14.3	19.0
	FineWeb-Edu	4.40	19.5	29.5	20.2	29.5	12.9	20.3	24.4	14.2	18.0
	FineWeb	4.60	22.8	32.1	21.5	32.1	12.6	16.4	20.2	16.2	20.1
	Dolma	4.60	22.0	34.0	22.7	34.9	12.8	17.6	22.0	17.3	22.2
	C4	4.60	20.7	29.2	19.3	28.7	11.6	14.3	17.3	14.9	18.8
300M	DCLM	4.10	23.3	8.3	24.0	7.0	39.6	53.5	45.9	54.7	45.6
	FineWeb-Edu	3.85	22.8	6.7	22.6	4.8	40.0	59.9	47.9	52.3	50.0
	FineWeb	4.10	23.1	7.1	22.4	5.3	40.8	61.2	46.4	55.6	46.6
	Dolma	3.95	22.3	9.8	22.3	6.5	44.4	49.3	52.3	50.5	47.4
	C4	4.00	26.4	8.5	25.3	6.5	41.8	64.7	55.5	59.4	51.9

Table 1 is the primary empirical result. It asks: for the hardest common validation-loss target reached by every listed method in all three seeds, what fraction of target-arrival tokens does MatrixPolicy save, and how many measured target-arrival minutes does each method require? Across all five datasets under the 100M-token budget, MatrixPolicy saves 19.5–22.8% of the SiLU with AdamW target-arrival tokens and 29.2–34.0% of the SiLU with Muon target-arrival tokens. Under the 300M-token budget, after longer training compresses endpoint gaps, MatrixPolicy still saves 22.3–26.4% versus SiLU with AdamW and 6.7–9.8% versus SiLU with Muon, with lower target-arrival time in every listed comparison.

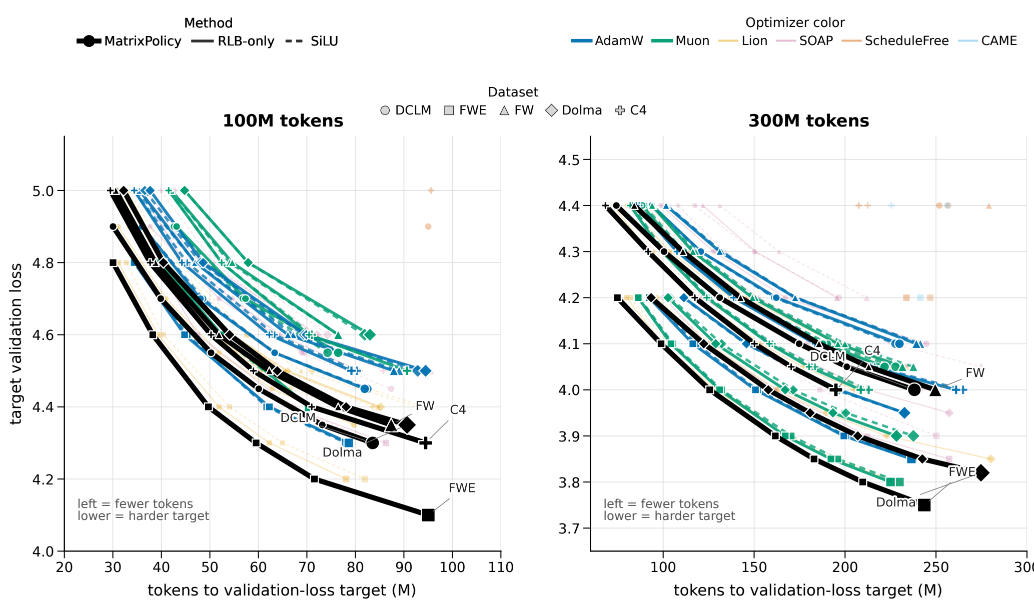

Figure 3: Target-arrival map for the completed fixed-scale study. Each point is the mean token count needed by one method to reach one validation-loss target on one dataset; connected points trace the same method and dataset from easier to harder targets. The horizontal coordinate is target-arrival tokens, and the vertical coordinate is the target validation loss, so curves further left reach the same loss with fewer tokens. The hard-target timing and token-saving percentages are quantified in Table 1.

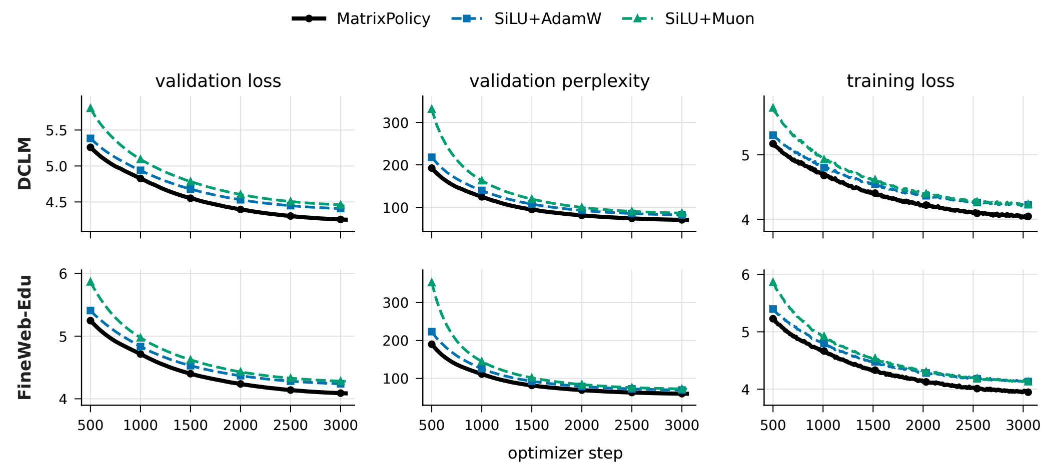

Figure 4: Representative 100M-token-budget training dynamics on DCLM and FineWeb-Edu. Rows show datasets and columns show validation loss, validation perplexity, and training loss. Lines are seed means over three runs and bands show one sample standard deviation. This figure uses the two standard SiLU baselines so that the early target-arrival effect in Table 1 remains visually legible.

The 100M-token-budget curves show why target arrival is the right main readout. MatrixPolicy separates early in validation loss and validation perplexity while keeping training loss competitive. A long fixed budget eventually compresses many activation/optimizer endpoints, especially under the 300M-token budget, so the paper emphasizes the number of tokens and minutes needed to reach the same useful validation-loss levels.

4.2 TODO: Larger-Scale Training Results

TODO: Insert the larger-scale training result here.

4.3 TODO: Component Ablations

TODO: Insert the component ablation study here.

5 Conclusion

RLB turns the nonlinear Transformer sublayer into an optimizer-visible interface by exposing grouped normalized coordinates, trainable groupwise rational responses, and an idealized positive matrix-pair relation used as a bounded balancing coordinate. MatrixPolicy uses that interface to update the RLB input and output matrices with role-aware statistics, while rational coefficients and non-RLB-matrix parameters use AdamW. The completed 100M-token and 300M-token runs show that MatrixPolicy reaches common validation losses with fewer tokens and lower measured target-arrival time than the listed SiLU and RLB controls. [todo: Complete this conclusion after the larger-scale and ablation results are added.]

References

Nicolas Boullé, Yuji Nakatsukasa, and Alex Townsend. Rational Neural Networks. In *Advances in Neural Information Processing Systems*, 2020. URL `https://proceedings.neurips.cc/paper/2020/hash/a3f390d88e4c41f2747bfa2f1b5f87db-Abstract.html`.

Xiangning Chen, Chen Liang, Da Huang, Esteban Real, Kaiyuan Wang, Hieu Pham, Xuanyi Dong, Thang Lu, Cho-Jui Hsieh, Yifeng Lu, and Quoc V. Le. Symbolic Discovery of Optimization Algorithms. In *Advances in Neural Information Processing Systems*,

2023. URL https://papers.neurips.cc/paper_files/paper/2023/hash/9a39b4925e35cf447ccba8757137d84f-Abstract-Conference.html.

Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, et al. PaLM: Scaling Language Modeling with Pathways. *arXiv preprint arXiv:2204.02311*, 2022. URL https://arxiv.org/abs/2204.02311.

Yann N. Dauphin, Angela Fan, Michael Auli, and David Grangier. Language Modeling with Gated Convolutional Networks. In *Proceedings of the 34th International Conference on Machine Learning*, pp. 933–941, 2017. URL https://proceedings.mlr.press/v70/dauphin17a.html.

Aaron Defazio, Xingyu Yang, Ahmed Khaled, Konstantin Mishchenko, Harsh Mehta, and Ashok Cutkosky. The Road Less Scheduled. In *Advances in Neural Information Processing Systems*, 2024. URL https://neurips.cc/virtual/2024/poster/96925.

Haishuo Fang, Ji-Ung Lee, Nafise Sadat Moosavi, and Iryna Gurevych. Transformers with Learnable Activation Functions. In *Findings of the Association for Computational Linguistics: EACL 2023*, 2023. URL https://aclanthology.org/2023.findings-eacl.181/.

Vineet Gupta, Tomer Koren, and Yoram Singer. Shampoo: Preconditioned Stochastic Tensor Optimization. In *Proceedings of the 35th International Conference on Machine Learning*, pp. 1842–1850, 2018. URL https://proceedings.mlr.press/v80/gupta18a.html.

Dan Hendrycks and Kevin Gimpel. Gaussian Error Linear Units (GELUs). *arXiv preprint arXiv:1606.08415*, 2016. URL https://arxiv.org/abs/1606.08415.

Diederik P. Kingma and Jimmy Ba. Adam: A Method for Stochastic Optimization. In *International Conference on Learning Representations*, 2015. URL https://mlanthology.org/iclr/2015/kingma2015iclr-adam/.

Jeffrey Li, Alex Fang, Georgios Smyrnis, Maor Ivgi, Matt Jordan, Samir Gadre, Hritik Bansal, Etash Guha, et al. DataComp-LM: In Search of the Next Generation of Training Sets for Language Models. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2024. doi: 10.52202/079017-0455. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/19e4ea30dded58259665db375885e412-Abstract-Datasets_and_Benchmarks_Track.html.

Kaizhao Liang, Lizhang Chen, Bo Liu, and Qiang Liu. Cautious Optimizers: Improving Training with One Line of Code. In *International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=zBPZeRjfgu.

Hong Liu, Zhiyuan Li, David Hall, Percy Liang, and Tengyu Ma. Sophia: A Scalable Stochastic Second-order Optimizer for Language Model Pre-training. In *International Conference on Learning Representations*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/06960915ba8674c7a898ec0b472b80ff-Abstract-Conference.html.

Jingyuan Liu, Jianlin Su, Xingcheng Yao, Zhejun Jiang, Guokun Lai, Yulun Du, Yidao Qin, Weixin Xu, Enzhe Lu, Junjie Yan, Yanru Chen, Huabin Zheng, Yibo Liu, Shaowei Liu, Bohong Yin, Weiran He, Han Zhu, Yuzhi Wang, Jianzhou Wang, Mengnan Dong, Zheng Zhang, Yongsheng Kang, Hao Zhang, Xinran Xu, Yutao Zhang, Yuxin Wu, Xinyu Zhou, and Zhilin Yang. Muon is Scalable for LLM Training. *arXiv preprint arXiv:2502.16982*, 2025. URL https://arxiv.org/abs/2502.16982.

Ilya Loshchilov and Frank Hutter. Decoupled Weight Decay Regularization. In *International Conference on Learning Representations*, 2019. URL https://openreview.net/forum?id=Bkg6RiCqY7.

Yang Luo, Xiaozhe Ren, Zangwei Zheng, Zhuo Jiang, Xin Jiang, and Yang You. CAME: Confidence-guided Adaptive Memory Efficient Optimization. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, pp. 4442–4453, 2023. doi: 10.18653/v1/2023.acl-long.243. URL https://aclanthology.org/2023.acl-long.243/.

Alejandro Molina, Patrick Schramowski, and Kristian Kersting. Padé Activation Units: End-to-End Learning of Flexible Activation Functions in Deep Networks. In *International Conference on Learning Representations*, 2020. URL https://openreview.net/forum?id=BJlBSkHtDS.

Guilherme Penedo, Hynek Kydlicek, Loubna Ben Allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro von Werra, and Thomas Wolf. The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2024. doi: 10.52202/079017-0970. URL https://papers.nips.cc/paper/2024/hash/370df50ccfdf8bde18f8f9c2d9151bda-Abstract-Datasets_and_Benchmarks_Track.html.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020. URL http://jmlr.org/papers/v21/20-074.html.

Prajit Ramachandran, Barret Zoph, and Quoc V. Le. Searching for Activation Functions. *arXiv preprint arXiv:1710.05941*, 2017. URL https://arxiv.org/abs/1710.05941.

Joel Schlotthauer, Christian Kroos, Chris Hinze, Viktor Hangya, Luzian Hahn, and Fabian Küch. Pre-Training LLMs on a Budget: A Comparison of Three Optimizers. *arXiv preprint arXiv:2507.08472*, 2025. URL https://arxiv.org/abs/2507.08472.

Andrei Semenov, Matteo Pagliardini, and Martin Jaggi. Benchmarking Optimizers for Large Language Model Pretraining. *arXiv preprint arXiv:2509.01440*, 2025. URL https://arxiv.org/abs/2509.01440.

Ishaan Shah, Anthony M. Polloreno, Karl Stratos, Philip Monk, Adarsh Chaluvaraju, Andrew Hojel, Andrew Ma, Anil Thomas, Ashish Tanwer, Darsh J. Shah, Khoi Nguyen, Kurt Smith, Michael Callahan, Michael Pust, Mohit Parmar, Peter Rushton, Platon Mazarakis, Ritvik Kapila, Saurabh Srivastava, Somanshu Singla, Tim Romanski, Yash Vanjani, and Ashish Vaswani. Practical Efficiency of Muon for Pretraining. *arXiv preprint arXiv:2505.02222*, 2025. URL https://arxiv.org/abs/2505.02222.

Noam Shazeer. GLU Variants Improve Transformer. *arXiv preprint arXiv:2002.05202*, 2020. URL https://arxiv.org/abs/2002.05202.

Noam Shazeer and Mitchell Stern. Adafactor: Adaptive Learning Rates with Sublinear Memory Cost. In *Proceedings of the 35th International Conference on Machine Learning*, pp. 4596–4604, 2018. URL https://proceedings.mlr.press/v80/shazeer18a.html.

Luca Soldaini, Rodney Kinney, Akshita Bhagia, Dustin Schwenk, David Atkinson, Russell Authur, Ben Bogin, Khyathi Chandu, et al. Dolma: An Open Corpus of Three Trillion Tokens for Language Model Pretraining Research. *arXiv preprint arXiv:2402.00159*, 2024. URL https://arxiv.org/abs/2402.00159.

Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced Transformer with Rotary Position Embedding. *arXiv preprint arXiv:2104.09864*, 2021. URL https://arxiv.org/abs/2104.09864.

Maosen Tang and Alex Townsend. Rational Neural Networks have Expressivity Advantages. *arXiv preprint arXiv:2602.12390*, 2026. doi: 10.48550/arXiv.2602.12390. URL https://arxiv.org/abs/2602.12390.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, et al. LLaMA: Open and Efficient Foundation Language Models. *arXiv preprint arXiv:2302.13971*, 2023. URL `https://arxiv.org/abs/2302.13971`.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention Is All You Need. In *Advances in Neural Information Processing Systems*, 2017. URL `https://papers.neurips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html`.

Nikhil Vyas, Depen Morwani, Rosie Zhao, Itai Shapira, David Brandfonbrener, Lucas Janson, and Sham Kakade. SOAP: Improving and Stabilizing Shampoo using Adam for Language Modeling. In *International Conference on Learning Representations*, 2025. URL `https://proceedings.iclr.cc/paper_files/paper/2025/hash/e988664070e9591f93fdcf605f7dc623-Abstract-Conference.html`.

Kaiyue Wen, David Leo Wright Hall, Tengyu Ma, and Percy Liang. Fantastic Pretraining Optimizers and Where to Find Them. In *International Conference on Learning Representations*, 2026. URL `https://openreview.net/forum?id=2J51qUZ0iG`.

Biao Zhang and Rico Sennrich. Root Mean Square Layer Normalization. *arXiv preprint arXiv:1910.07467*, 2019. URL `https://arxiv.org/abs/1910.07467`.

Yushun Zhang, Congliang Chen, Ziniu Li, Tian Ding, Chenwei Wu, Diederik P. Kingma, Yinyu Ye, Zhi-Quan Luo, and Ruoyu Sun. Adam-mini: Use Fewer Learning Rates To Gain More. In *International Conference on Learning Representations*, 2025. URL `https://proceedings.iclr.cc/paper_files/paper/2025/hash/45ae878717399e6f62d57c65f052cd46-Abstract-Conference.html`.

Jiawei Zhao, Zhenyu Zhang, Beidi Chen, Zhangyang Wang, Anima Anandkumar, and Yuandong Tian. GaLore: Memory-Efficient LLM Training by Gradient Low-Rank Projection. In *International Conference on Machine Learning*, 2024. URL `https://openreview.net/forum?id=hYHsrKDiX7`.

Rosie Zhao, Depen Morwani, David Brandfonbrener, Nikhil Vyas, and Sham Kakade. Deconstructing What Makes a Good Optimizer for Language Models. In *International Conference on Learning Representations*, 2025. URL `https://arxiv.org/abs/2407.07972`.

A EXPERIMENT APPENDIX

This appendix gives the fixed-scale protocol behind Section 4.1, the full training curves for the five corpora, and the broader optimizer endpoint comparison used to contextualize the main SiLU and RLB controls.

A.1 FIXED-SCALE TARGET-ARRIVAL STUDY

A.1.1 MODEL AND DATA PIPELINE

The fixed-scale experiments use a custom causal decoder Transformer. The block uses a LLaMA-style pre-normalized layout with RMSNorm before attention and before the nonlinear sublayer, bias-free query/key/value and output attention projections, rotary position embeddings, causal scaled-dot-product attention, residual attention and nonlinear-sublayer updates, a final RMSNorm, and a tied token-embedding/output head (Zhang & Sennrich, 2019; Su et al., 2021; Touvron et al., 2023). The fixed model has 12 layers, width 768, 12 heads of dimension 64, and context length 256.

The SiLU controls use a SwiGLU-like gated nonlinear sublayer with bias-free matrices $W_g, W_v \in \mathbb{R}^{768\times 2048}$ and $W_o \in \mathbb{R}^{2048\times 768}$, computing $(\mathrm{SiLU}(xW_g) \odot xW_v)W_o$. MatrixPolicy and RLB-only configurations use a matched single-branch matrix pair. An input matrix maps width 768 to 3072 latent coordinates, the coordinates are partitioned into groups and transformed by the real-pole-free P5/Q4 rational response, and an output matrix maps the 3072 coordinates back to width

768. With the default group size 256, this gives $G = 12$ groups of width $m = 256$; the RMS floor is $\epsilon_{\text{rms}} = 10^{-6}$. This is the RLB configuration used by the main MatrixPolicy and RLB-control results.

Tokenization uses the GPT-2 tokenizer through the Hugging Face tokenizer interface, without adding tokenizer-specific special tokens; an EOS or pad token is appended between documents. Training samples random contiguous blocks of length 257 and forms next-token prediction pairs of length 256. The DCLM, FineWeb-Edu, FineWeb, Dolma-sample, and C4 runs use the text field specified in the manifests, with validation slices drawn from held-out token offsets of the same corpus stream.

A.1.2 Training Protocol

All runs use batch size 16, gradient accumulation 2, and therefore 32768 global tokens per optimizer step. The 100M-token budget consists of 3050 optimizer steps; the 300M-token budget consists of 9150 optimizer steps. Each method/dataset pair is run with seeds 1337, 2027, and 3407. Validation is performed every 50 optimizer steps, so token-to-target arrivals are quantized in increments of 1.64M tokens.

The MatrixPolicy rows in the main table use the RLB activation, whose active nonlinearity is the groupwise P5/Q4 response in equations 3–5. The evaluated configuration uses AdamW as the backbone optimizer and disables the separate rational-coefficient optimizer. It enables MatrixPolicy group redistribution with gain/pressure/activity strengths $0.20/0.10/0.20$, schedule window 0.02–0.30, and group multiplier clip $[0.75, 1.35]$. The RLB matrix pair then uses the role/depth-scaled AdamW branch and the early Muon substep from Section 3.2. The bounded RLB pair-rescale wrapper runs every five optimizer steps with covariant optimizer-state scaling. Rational coefficients, attention weights, embeddings, normalization parameters, and ordinary Transformer weights use AdamW in this configuration.

Target-arrival minutes in Table 1 are computed seed-wise from the first validation event that reaches the shared target and the same run's measured seconds per optimizer step. The per-step time is the training-script wall time recorded in the run summary divided by completed optimizer steps for that run; target arrivals are therefore measured with the same evaluation cadence and timing source across methods.

A.1.3 SUPPORTING FIXED-SCALE CURVES

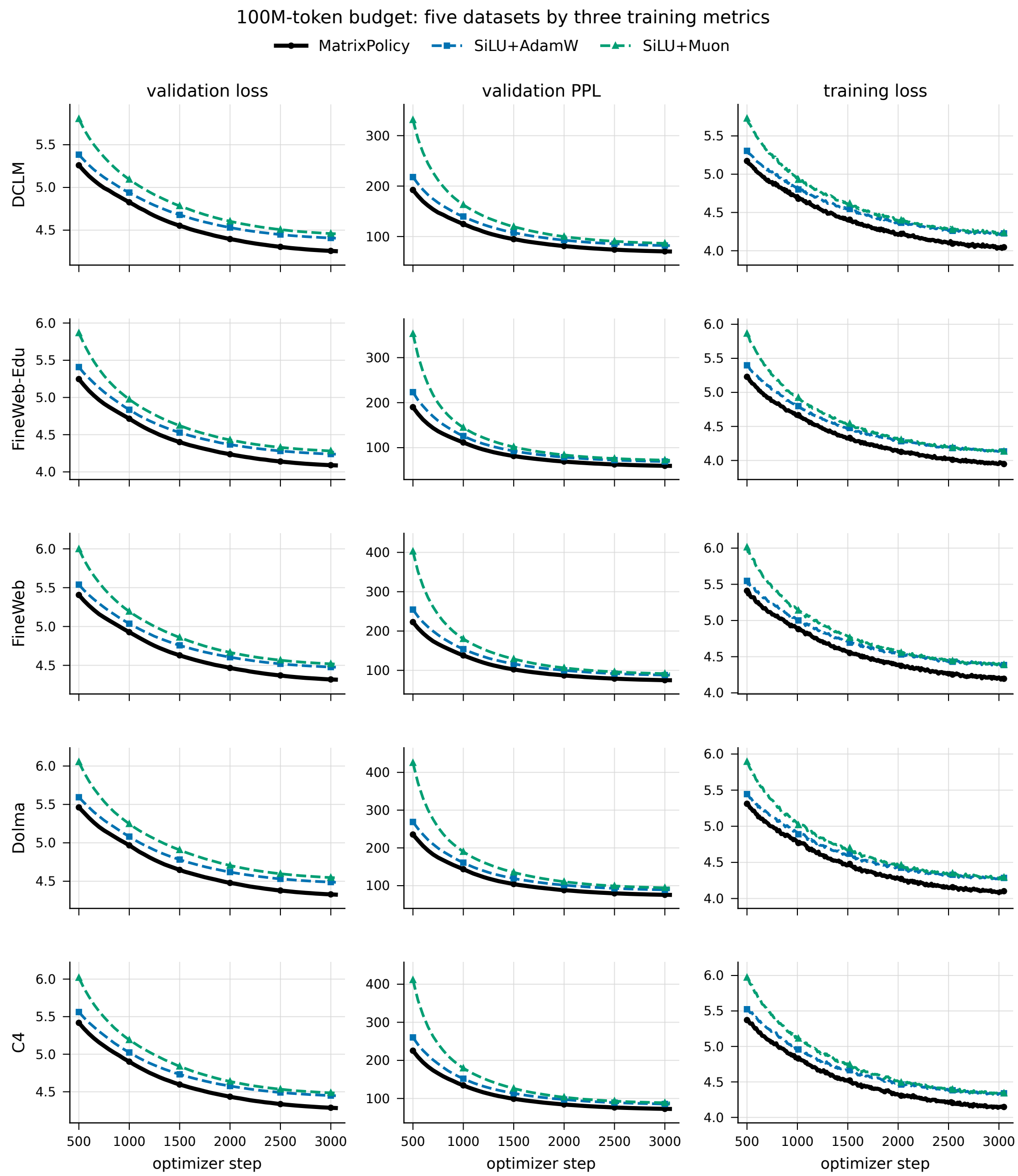

Figure 5: 100M-token-budget dynamics across all five datasets. Rows are datasets and columns are validation loss, validation perplexity, and training loss. Lines are seed means over three runs and bands show one sample standard deviation.

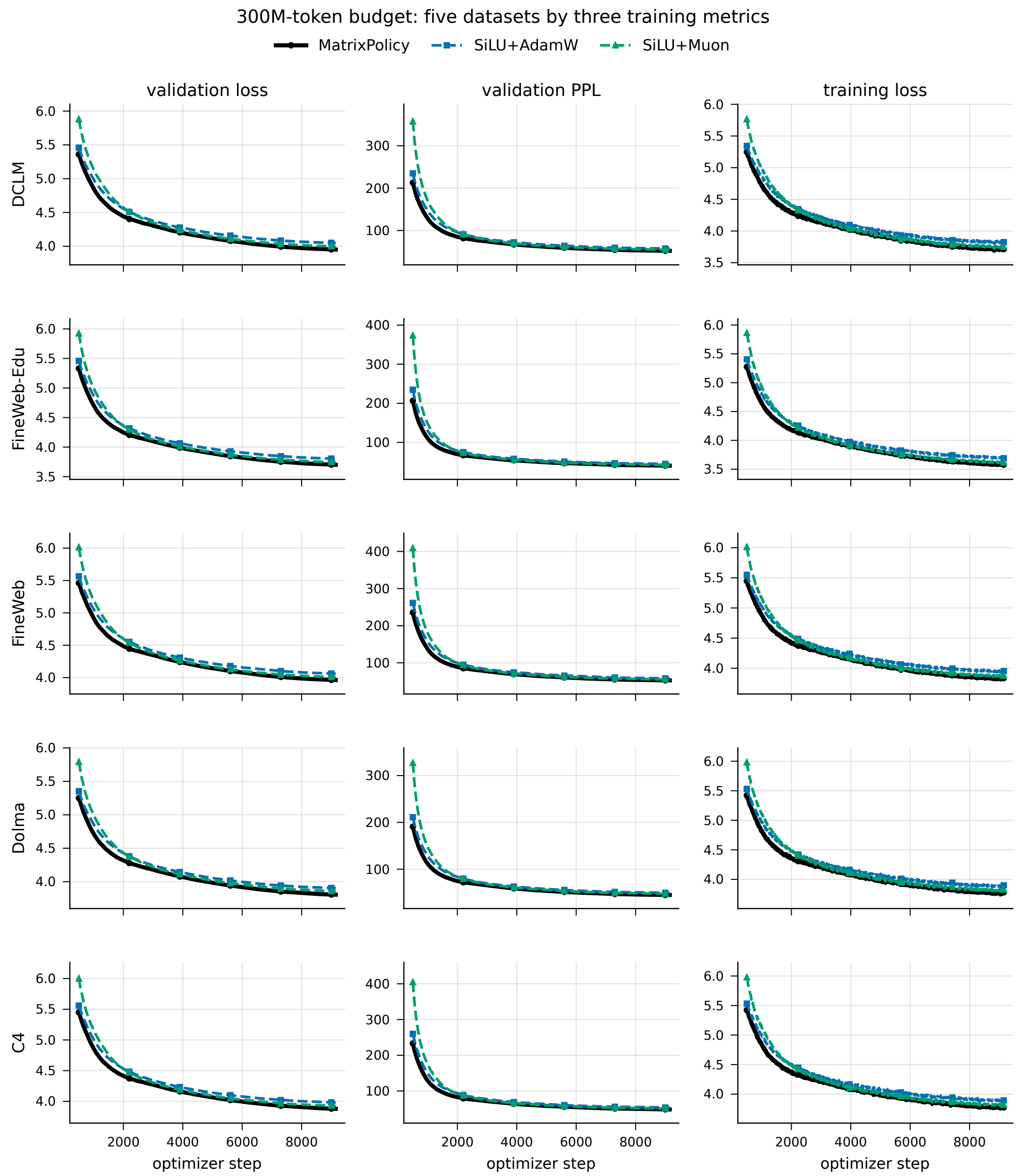

Figure 6: 300M-token-budget dynamics across all five datasets. Rows are datasets and columns are validation loss, validation perplexity, and training loss. Lines are seed means over three runs and bands show one sample standard deviation.

A.1.4 Broad Optimizer Endpoint Table

The fixed-scale package also includes the broader optimizer endpoints referenced from Section 4.1. These rows use the same model, datasets, seeds, token budgets, and validation schedule as the main comparison, while replacing the nonlinear-sublayer optimizer according to the table entry.

Table 2: Final validation loss for the broader optimizer sweep on the fixed 12-layer, width-768 language model. Values are mean ± sample standard deviation over three seeds; lower is better. RLB rows use the same rational latent-basis activation without MatrixPolicy.

Budget	Method	DCLM	FineWeb-Edu	FineWeb	Dolma	C4
100M tokens	MatrixPolicy	4.2538±0.0063	4.0873±0.0102	4.3162±0.0125	4.3253±0.0053	4.2837±0.0197
	SiLU + AdamW	4.4056±0.0099	4.2375±0.0086	4.4758±0.0097	4.4862±0.0012	4.4469±0.0160
	SiLU + Lion	4.3183±0.0069	4.1494±0.0092	4.3825±0.0083	4.3878±0.0046	4.3536±0.0156
	SiLU + Muon	4.4572±0.0126	4.2787±0.0243	4.5163±0.0264	4.5443±0.0108	4.4824±0.0233
	SiLU + SOAP	4.4160±0.0038	4.2630±0.0220	4.4840±0.0112	4.4987±0.0035	4.4582±0.0146
	SiLU + ScheduleFree	4.9023±0.0111	4.8258±0.0072	5.0142±0.0188	5.0494±0.0149	5.0064±0.0160
	SiLU + CAME	5.0107±0.0143	4.9207±0.0123	5.1325±0.0126	5.1650±0.0189	5.1230±0.0178
	RLB + AdamW	4.4046±0.0081	4.2392±0.0077	4.4717±0.0138	4.4878±0.0027	4.4412±0.0162
	RLB + Lion	4.2946±0.0083	4.1361±0.0083	4.3626±0.0112	4.3622±0.0066	4.3271±0.0160
	RLB + Muon	4.4674±0.0121	4.2831±0.0284	4.5157±0.0059	4.5476±0.0153	4.4764±0.0099
	RLB + SOAP	4.4360±0.0442	4.2747±0.0238	4.6893±0.3359	4.5183±0.0363	4.4458±0.0190
	RLB + ScheduleFree	4.8871±0.0046	4.7957±0.0133	4.9993±0.0192	5.0363±0.0129	4.9842±0.0164
	RLB + CAME	5.0139±0.0086	4.9150±0.0063	5.1340±0.0162	5.1514±0.0168	5.1109±0.0155
300M tokens	MatrixPolicy	3.9518±0.0282	3.7015±0.0212	3.9623±0.0081	3.8062±0.0073	3.8777±0.0144
	SiLU + AdamW	4.0493±0.0275	3.8035±0.0182	4.0612±0.0101	3.9037±0.0091	3.9811±0.0128
	SiLU + Lion	3.9934±0.0230	3.7440±0.0208	4.0015±0.0085	3.8475±0.0094	3.9213±0.0105
	SiLU + Muon	3.9973±0.0305	3.7454±0.0170	4.0066±0.0128	3.8581±0.0101	3.9251±0.0134
	SiLU + SOAP	4.0964±0.0300	3.8629±0.0201	4.1139±0.0101	3.9568±0.0093	4.0349±0.0108
	SiLU + ScheduleFree	4.3657±0.0298	4.1559±0.0238	4.3979±0.0106	4.2151±0.0053	4.3163±0.0107
	SiLU + CAME	4.3682±0.0226	4.1503±0.0211	4.4060±0.0189	4.2492±0.0270	4.3298±0.0148
	RLB + AdamW	4.0496±0.0298	3.8024±0.0206	4.0601±0.0110	3.9045±0.0082	3.9787±0.0098
	RLB + Lion	3.9887±0.0295	3.7416±0.0214	3.9960±0.0105	3.8412±0.0085	3.9132±0.0139
	RLB + Muon	3.9913±0.0260	3.7373±0.0187	3.9991±0.0110	3.8478±0.0047	3.9189±0.0149
	RLB + SOAP	4.0608±0.0331	3.8240±0.0128	4.1081±0.0320	3.9269±0.0125	4.0024±0.0194
	RLB + ScheduleFree	4.3607±0.0344	4.1421±0.0217	4.3864±0.0081	4.2121±0.0112	4.3084±0.0071
	RLB + CAME	4.4503±0.0426	4.2255±0.0358	4.4804±0.0064	4.2665±0.0409	4.3651±0.0582

A.2 TODO: Larger-Scale Training Results

TODO: Insert larger-scale training details here.

A.3 TODO: Component Ablations

TODO: Insert component ablation details here.

B Rational Latent Basis Calculus

This section records the identities behind the RLB quantities used by MatrixPolicy. All calculations are local to one layer and one group unless otherwise stated, so we suppress (l, g) in the notation. The response parameterization first guarantees that the denominator cannot vanish on the real line:

$$\phi_1(u) = |u|, \quad \phi_2(u) = u^2, \quad \phi_3(u) = |u|^3, \quad \phi_4(u) = u^4,$$
$$P(u) = \sum_{i=0}^{5} a_i u^i, \qquad Q(u) = 1 + \sum_{j=1}^{4} |b_j| \phi_j(u). \tag{35}$$

Since $\phi_j(u) \geq 0$ for all real u, $Q(u) \geq 1$ and $R(u) = P(u)/Q(u)$ has no real pole. The same parameterization keeps the curve derivatives visible to the optimizer. For all $u \neq 0$, the input derivative is

$$R'(u) = \frac{P'(u)Q(u) - P(u)Q'(u)}{Q(u)^2}, \tag{36}$$

where

$$P'(u) = \sum_{i=1}^{5} i a_i u^{i-1}, \qquad Q'(u) = |b_1| \operatorname{sign}(u) + 2|b_2|u + 3|b_3|u|u| + 4|b_4|u^3. \tag{37}$$

Derivative-gain statistics use the autodiff convention $\operatorname{sign}(0) = 0$ at the absolute-value kink. The coefficient features are explicit:

$$\frac{\partial R(u)}{\partial a_i} = \frac{u^i}{Q(u)}, \qquad \frac{\partial R(u)}{\partial b_j} = -\frac{P(u)}{Q(u)^2} \operatorname{sign}(b_j)\phi_j(u), \tag{38}$$

where $\operatorname{sign}(0) = 0$ is also used at $b_j = 0$. The response factor $R(u)$, the input derivative $R'(u)$, and the coefficient features in equation 38 are all observable from the group-specific rational response. For the evaluated groupwise P5/Q4 response, the coefficient set in equation 12 is $\Theta^R_{l,g} = \{a_{l,g,\cdot}, b_{l,g,\cdot}\}$; the more general block definition averages over the trainable rational-response coefficient blocks present in a group. These coefficient derivatives are classical away from $b_j = 0$ and use the stated autodiff subgradient convention at $b_j = 0$.

The RMS-normalized group map turns these curve quantities into matrix-role signals. For a group vector $z \in \mathbb{R}^m$, define

$$r(z) = \sqrt{m^{-1}\|z\|_2^2 + \epsilon_{\mathrm{rms}}}, \qquad u(z) = z/r(z), \qquad h(z) = r(z)R(u(z)), \tag{39}$$

where R is applied coordinatewise. The following differentials are taken on the domain $r(z) > 0$, equivalently $\epsilon_{\mathrm{rms}} > 0$ or $\epsilon_{\mathrm{rms}} = 0$ with $z \neq 0$. The radius derivative is

$$\mathrm{d}r = \frac{z^\top \mathrm{d}z}{mr} = \frac{u^\top \mathrm{d}z}{m}, \tag{40}$$

and the normalized-coordinate derivative is

$$\mathrm{d}u = \frac{1}{r}\left(I - \frac{uu^\top}{m}\right)\mathrm{d}z. \tag{41}$$

The factor $I - uu^\top/m$ is exact in equation 41; it becomes the orthogonal tangent projector to the RMS sphere only in the unfloored case where $\epsilon_{\mathrm{rms}} = 0$ and $z \neq 0$. Away from absolute-value kinks of the scalar curve, the group Jacobian is

$$J_h(z) := \frac{\partial h}{\partial z} = R(u)\frac{u^\top}{m} + \operatorname{diag}(R'(u))\left(I - \frac{uu^\top}{m}\right). \tag{42}$$

At nonsmooth coordinates, equation 42 uses the same autodiff subgradient convention as equation 36 and equation 38. The response vector $R(u)$ controls the curve factor inside the realized feature $h = rR(u)$, while $R'(u)$ appears in the normalized-coordinate component of the input Jacobian. The MatrixPolicy summaries $\hat{o}$ and $\hat{d}$ are therefore partial role signals: $\hat{o}$ omits the group radius r, and $\hat{d}$ omits the radial term $R(u)u^\top/m$, the output block $B_{l,g}$, and the upstream gradient. These equations expose a compact set of role signals for the adjacent matrices without requiring a full block-Jacobian preconditioner.

The same normalization also exposes a positive matrix-pair coordinate. Set $\epsilon_{\mathrm{rms}} = 0$ and take $z \neq 0$ and $\lambda > 0$. Then

$$r(\lambda z) = \lambda r(z), \qquad u(\lambda z) = u(z), \qquad h(\lambda z) = \lambda h(z). \tag{43}$$

Scaling $A_{l,g}$ by λ and the matching column block $B_{l,g}$ by λ^{-1} therefore leaves the group contribution unchanged. Applying this independently to all groups gives equation 9.

With $\epsilon_{\mathrm{rms}} > 0$, define the floored maps

$$r_\epsilon(z) = \sqrt{m^{-1}\|z\|_2^2 + \epsilon_{\mathrm{rms}}}, \qquad u_\epsilon(z) = z/r_\epsilon(z), \qquad h_\epsilon(z) = r_\epsilon(z)R(u_\epsilon(z)). \tag{44}$$

For $\rho^2 = m^{-1}\|z\|_2^2$,

$$u_\epsilon(\lambda z) = \kappa(\lambda, z)u_\epsilon(z), \qquad \kappa(\lambda, z) = \frac{\lambda\sqrt{\rho^2 + \epsilon_{\mathrm{rms}}}}{\sqrt{\lambda^2\rho^2 + \epsilon_{\mathrm{rms}}}}. \tag{45}$$

Let L_R be a Lipschitz constant of the scalar curve over an interval containing all coordinates of $u_\epsilon(z)$ and $\kappa(\lambda, z)u_\epsilon(z)$. Applying this scalar Lipschitz bound coordinatewise, and then compensating the output matrix by λ^{-1}, gives

$$\left\|\lambda^{-1}h_\epsilon(\lambda z) - h_\epsilon(z)\right\|_2 \le \left|\frac{r_\epsilon(\lambda z)}{\lambda} - r_\epsilon(z)\right| \|R(u_\epsilon(z))\|_2 \tag{46}$$

$$+ \frac{r_\epsilon(\lambda z)}{\lambda} L_R |\kappa(\lambda, z) - 1| \, \|u_\epsilon(z)\|_2 . \tag{47}$$

Both terms vanish when $\epsilon_{\mathrm{rms}} = 0$. With a bounded rescaling $|\log\lambda| \le c$, the same bound is small when $e^{2c}\epsilon_{\mathrm{rms}}/\rho^2$ is small and the scalar response and derivative are bounded on the compact interval above. Near the floor, or under an arbitrarily large reciprocal rescaling, the positive pair-rescaling identity no longer gives a useful invariance. The wrapper therefore uses this direction only through a clipped log step with floored matrix RMS values and matching optimizer-state rescaling.

C MatrixPolicy Update Geometry

MatrixPolicy uses the RLB calculus because the two adjacent matrices see different local factors. For one training example, let $\bar{y}_l = \nabla_{y_l}\mathcal{L}$ be the upstream gradient, and let $J_{l,g} = \partial h_{l,g}/\partial z_{l,g}$ be the group Jacobian in equation 42. The adjacent matrix gradients factor as

$$\nabla_{B_{l,g}}\mathcal{L} = \bar{y}_l h_{l,g}^\top, \tag{48}$$

$$\nabla_{A_{l,g}}\mathcal{L} = \left(J_{l,g}^\top B_{l,g}^\top \bar{y}_l\right) x_l^\top . \tag{49}$$

The output matrix is coupled directly to the realized feature $h_{l,g}$, whereas the input matrix is coupled to the tangent and radial components of the RLB Jacobian before the upstream gradient reaches x_l. Minibatch and sequence gradients sum these factors over positions and examples. Response RMS summaries therefore align with the output role, and derivative RMS summaries align with the input role. They remain proxies because the full output feature contains the group radius $r(z)$, and the full input gradient depends on $B_{l,g}$, the upstream gradient, and the radial term in equation 42.

The optimizer step consumes these role signals in the same order as Section 3.2. Backpropagation first produces gradients with respect to the pre-update parameters. Pressure and activity statistics are then computed from these gradients, A_l^t, B_l^t, and the rational coefficient set $\Theta_{l,g}^R$ before any parameter is changed. With EMA decay $\beta = 0.95$, the implemented statistics follow the same recurrence as equation 13,

$$q_{l,g,t}^{\mathrm{in}} = \beta q_{l,g,t-1}^{\mathrm{in}} + (1-\beta) p_{l,g,t}^{\mathrm{in}}, \tag{50}$$

$$q_{l,g,t}^{\mathrm{out}} = \beta q_{l,g,t-1}^{\mathrm{out}} + (1-\beta) p_{l,g,t}^{\mathrm{out}}, \tag{51}$$

$$q_{l,g,t}^{R} = \beta q_{l,g,t-1}^{R} + (1-\beta) p_{l,g,t}^{R}. \tag{52}$$

The first observed post-backpropagation values initialize these EMA states. Live curve gains are read from the RLB forward-statistics hook, which stores subsampled per-group response and derivative RMS summaries with the ϵ_{live} floor, and enter the group-gradient multipliers in equation 16. The fixed-grid probe gains in equation 28 use the $\epsilon_{\mathrm{probe}}$ floor and enter only the pair-rescale target. The Muon fraction in equation 26 and the balancing quantities in equations 30–31 use pre-update pressure/activity EMAs. Non-matrix Transformer parameters and rational coefficients follow AdamW. The RLB matrix blocks use the scaled gradients in equation 21 for the sequential AdamW and Muon matrix branches in equation 27. After the AdamW/Muon matrix branch, the on-policy wrapper applies the positive matrix-pair move in equation 34 on scheduled rescaling steps, and scales first- and second-moment-like optimizer-state buffers in the matching direction.

The pressure signal is a stable magnitude proxy for motion along the input/output balance direction. In the idealized unfloored rescaling direction, an infinitesimal increase of the input representative and reciprocal decrease of the output representative has the loss derivative

$$\Delta_g = \left\langle \nabla_{A_{l,g}}\mathcal{L}, A_{l,g}\right\rangle_F - \left\langle \nabla_{B_{l,g}}\mathcal{L}, B_{l,g}\right\rangle_F . \tag{53}$$

MatrixPolicy does not estimate Δ_g directly. Instead it uses RMS relative-gradient proxies. For same-shaped nonzero unfloored $W, \mathcal{G}$, if $W^+ = W - \eta\mathcal{G}$ and $\eta\|\mathcal{G}\|_F < c\|W\|_F$ for a fixed $c < 1$, then

$$\log \mathrm{rms}(W^+) - \log \mathrm{rms}(W) = -\eta \frac{\langle \mathcal{G}, W\rangle_F}{\|W\|_F^2} + O\left(\eta^2 \frac{\|\mathcal{G}\|_F^2}{\|W\|_F^2}\right), \tag{54}$$

and Cauchy–Schwarz implies

$$\left|\frac{\langle \mathcal{G}, W\rangle_F}{\|W\|_F^2}\right| \le \frac{\|\mathcal{G}\|_F}{\|W\|_F} = \frac{\operatorname{rms}(\mathcal{G})}{\operatorname{rms}(W)}. \tag{55}$$

The implemented quantities p^{in} and p^{out} replace the unfloored ratio on the right-hand side by floored RMS proxies. The floor makes the signals stable near zero, but it also means they are no longer exact upper bounds on $\operatorname{rms}(\mathcal{G})/\operatorname{rms}(W)$ near the floor. Thus $\pi_{l,g} = \log q^{\text{in}}_{l,g} - \log q^{\text{out}}_{l,g}$ is an RMS-imbalance proxy. Its sign identifies which role has the larger EMA of the floored proxy, and its clipped magnitude supplies the pressure term used by equation 18 and equation 30.

The gain and pressure rules in equations 17–18 are log-space redistribution rules. High derivative gain reduces the input-role multiplier for that group, high response gain reduces the output-role multiplier, and a positive $\pi_{l,g}$ reduces the input-role multiplier while increasing the output-role multiplier. Coefficient activity in equation 19 damps groups whose rational curve is changing faster than the adjacent matrix pair. These corrections would change the layer-wide step scale unless they were centered. Before clipping,

$$\widehat{c}_{l,g,\sigma} = \frac{c_{l,g,\sigma}}{\operatorname{geomean}_{j=1}^{G} c_{l,j,\sigma}}, \qquad \frac{1}{G}\sum_{g=1}^{G} \log \widehat{c}_{l,g,\sigma} = 0. \tag{56}$$

The policy redistributes update scale across groups while preserving the role-wise mean log multiplier before clipping. The subsequent clip in equation 20 bounds noisy group statistics and can break exact centering.

The pair-rescale target is motivated by equalizing representative input/output role scales in the positive log coordinate. For the unfloored nonzero-matrix calculation, write

$$\gamma_{l,g} = \log \operatorname{rms}(A_{l,g}) - \log \operatorname{rms}(B_{l,g}). \tag{57}$$

An active rescaling step applies $A_{l,g} \leftarrow e^{\ell_{l,g,t}} A_{l,g}$ and $B_{l,g} \leftarrow e^{-\ell_{l,g,t}} B_{l,g}$, hence

$$\gamma^{+}_{l,g} = \gamma_{l,g} + 2\ell_{l,g,t}. \tag{58}$$

If clipping is ignored and $\lambda_{l,g,t} = s_{l,t}\chi^{\text{rs}}_{l,g}$ uses the schedule and activity factor from equations 31–32, then equation 33 gives

$$\gamma^{+}_{l,g} = (1 - \lambda_{l,g,t})\gamma_{l,g} + \lambda_{l,g,t}\tau_{l,g}. \tag{59}$$

Because $0 \le s_{l,t} \le 0.625$ and $\chi^{\text{rs}}_{l,g} \le \exp(0.125)$ under the stated clips, $0 \le \lambda_{l,g,t} < 1$ before log-step clipping. Thus this is an exact contraction only in the nonzero-matrix, unfloored matrix-RMS, unclipped idealization. Separately, the function-preserving interpretation of the same positive pair direction also uses the unfloored activation map $\epsilon_{\text{rms}} = 0$ from Appendix B. The implementation uses the floored proxy in equation 29; near the matrix floor, the log-ratio interpretation and the contraction model are approximate.

In the representative model that ignores group-dependent orientation, radius, and upstream-gradient constants, reciprocal rescaling makes the input-side role scale vary as $\hat{d}^{\text{probe}} e^{-\gamma}$ and the output-side role scale vary as $\hat{o}^{\text{probe}} e^{\gamma}$. Equalizing these two log scales gives $\gamma^{\star} = \frac{1}{2}(\log \hat{d}^{\text{probe}} - \log \hat{o}^{\text{probe}})$. The wrapper target adds a smaller clipped RMS-pressure bias,

$$\tau_{l,g} = \frac{1}{2}\left(\log \hat{d}^{\text{probe}}_{l,g} - \log \hat{o}^{\text{probe}}_{l,g}\right) + 0.25\,\bar{\pi}_{l,g}, \qquad \bar{\pi}_{l,g} = \operatorname{clip}(\pi_{l,g}, -1.25, 1.25) \tag{60}$$

using fixed-grid probe gains from equation 28. Probe gains tie the balance target to the current curve shape, while the $\bar{\pi}$ term lets recent matrix-pressure imbalance shift the target within a bounded range.

The wrapper's final pair-rescale operation follows the approximate representative geometry above. The AdamW/Muon branch in equation 27 updates the represented map through the scaled matrix gradients. The time window inside the Muon fraction in equation 26 is

$$w(\xi_t) = \operatorname{smoothstep}(0.02, 0.12, \xi_t)\left[1 - \operatorname{smoothstep}(0.20, 0.36, \xi_t)\right]. \tag{61}$$

Together with the role factor $\varrho_\sigma(l)$, penalty $\psi_{l,t}$, and clipping in the full $\mu_{l,\sigma,t}$ formula, this window confines the matrix-direction update to a fixed early interval. The penalty $\psi_{l,t}$ reduces this branch when RMS imbalance or coefficient-gradient activity is high. Muon enters as a bounded, state-dependent matrix-direction substep for the RLB matrix pair.